Design Decisions and Rationale

The design of the SmartClinic database centered on accurately modelling the clinic's real-world entities and relationships: Patients, Staff, Doctor, Appointments, Payments, and Treatment & Medicine. The most significant design decision was the use of an EER super-class/sub-class relationship to represent staff specialization, with Staff as the super-type and Doctor as a sub-type. This inheritance structure was chosen because doctors share all general staff attributes but also require additional, role-specific information (such as specialization). Modelling this as inheritance avoided duplicating shared attributes and kept the schema normalized, while still allowing Doctor-specific data to be stored separately. This structure also enabled cascade-delete relationships between Treatments and Appointments, so that dependent records are automatically cleaned up when a parent record is removed, preserving referential integrity without requiring manual cleanup logic.

Team Issues Encountered and Resolutions

During implementation, the team encountered two main technical issues, both resolved collaboratively. First, the DDL script produced recursive deletion errors between related entities (patients, doctors, appointments). This was traced back to the order in which tables were created in MySQL Workbench: child tables were being created before their parent tables existed. The fix was to reorder the script so that parent tables (payment, staff, patients) were created before dependent/inherited entities such as doctor. Second, a data type error arose because the DoctorId attribute in the Appointment relation was linked to the general Staff entity rather than to the Doctor sub-entity. This was resolved by remodelling the foreign key to reference doctors.doctor_id directly, correctly reflecting that only doctors, not all staff, can be assigned appointments.

Utilization of AI Tools

AI tools were used in a supporting, not authoring, role during the project. They were used to help diagnose the cause of the recursive deletion and foreign-key errors described above by explaining MySQL error messages and suggesting likely causes related to table creation order and key references, which the team then verified and applied manually in MySQL Workbench. AI assistance was also used to review the clarity and structure of report drafts. All AI-suggested fixes were tested against the actual database before being adopted, and no AI-generated SQL or schema content was used without the team's review and understanding.

Suggested Future Technical Enhancements

Beyond the planned view and trigger for appointment reporting and transaction logging, the schema could be extended with stored procedures for common operations (e.g., booking an appointment or processing a payment) to enforce business logic consistently at the database level. Adding indexes on frequently queried foreign keys (such as patient_id and doctor_id) would improve performance as data volume grows. An audit-logging table could track changes to sensitive records like payments and treatments. Finally, introducing role-based access control at the database level would strengthen security by restricting which staff roles can view or modify specific tables.